2026-10-23 23:22:21 You

So i wanna make an optics eye wear website for my father's business, since we have a popular optic busines named mehar for eyes, i have made websites before but not like this one, secondly the webstie would be more like a landing page or what u call a visiting website not a ecommerce buyign and selling, i mean the buying and selling option will be added but not now it will be for later, for now we will display our products on the website but all there ordering will be done on the official whatsapp number of mehar for eyes, so suggest e ideas, gimme best prompts so i cna give commands to my cursor and build the best out of it and no ai slop looks

Claude

Blunt take first: **real photos make or break this.** Stock photos and AI-generated images are what make optical sites look like slop. Spend one afternoon shooting the actual frames on a plain surface in daylight, plus the shop front. Everything below assumes you do that.

Direction

Stack: Next.js (App Router) + TypeScript + Tailwind. Products live in one typed `products.ts` file now. When ecommerce comes later, you swap that file for a DB or API and nothing else changes.

Audience reality: Almost all traffic will be phones on mediocre mobile data. Optimize images hard, keep the WhatsApp button always reachable, and consider English + Urdu later.

Look (the anti-slop part):

- Warm off-white background, near-black text, one accent (deep tortoiseshell brown, bottle green, or burgundy). No purple/blue gradients.
- Type pairing: a serif display font (Fraunces or Cormorant Garamond) with a clean sans (DM Sans or Manrope). Not Inter everywhere.
- Eyewear-specific details: circular lens-shaped image masks, thin hairline dividers, big product photos with lots of whitespace.
- No glassmorphism, no emoji icons, no "Welcome to our website" copy, no fake testimonials, no lorem ipsum.

Pages:

1. Home: hero, featured frames, categories, why Mehar for Eyes, shop location and hours, sticky WhatsApp CTA
2. Collection: filter by category (men / women / kids / sunglasses / lenses), gender, frame shape
3. Product detail: gallery, details, "Order on WhatsApp" button with a prefilled message
4. About / Visit us: story, address, map, hours, eye-test info if offered

Prompts for Cursor

Step 0: create `.cursor/rules/mehar.mdc` (always apply):

```
</> Plain text

Project: Mehar for Eyes, a showcase website for an established optical business in Pakistan.
Not ecommerce yet. All orders happen via WhatsApp. Ecommerce will be added later, so keep
product data separate from UI.

Stack: Next.js App Router, TypeScript, Tailwind CSS, next/image. No UI component libraries
unless I ask.

Design rules:
- Warm off-white background (#FAF7F2), near-black text (#1A1A1A), one accent color (#5A2E1F).
No gradients, no glassmorphism, no purple/blue.
- Fonts via next/font: Fraunces (headings), DM Sans (body).
- Generous whitespace, large product imagery, hairline borders (1px), subtle motion only
(fade/translate on scroll, 200-300ms).
- No emojis in UI. No lorem ipsum. No generic marketing copy like "Welcome to" or "Discover
the best". Write short, specific, human copy.
- Mobile first. Must look correct at 360px width.
- Accessible: alt text on all images, visible focus states, contrast AA.
- Performance: all images through next/image with sizes, lazy load below the fold.

Code rules:
- Products come only from /data/products.ts. UI never hardcodes product info.
- WhatsApp link logic lives only in /lib/whatsapp.ts.
- Give full files, not snippets.
```

Prompt 1: foundation

```
</> Plain text

Set up the project structure: app/ with routes /, /collection, /collection/[slug], /about.
Create /data/products.ts with a typed Product (id, slug, name, brand, category, gender,
frameShape, material, color, price optional, images[], description, inStock). Add 6
placeholder products clearly marked TODO so I can replace them. Create /lib/whatsapp.ts
exporting buildWhatsAppLink(message) using wa.me with number from env
NEXT_PUBLIC_WHATSAPP_NUMBER (format 92XXXXXXXXXX, no + or leading 0) and encodeURIComponent.
```

Add the layout with fonts, header (logo text, nav: Collection, About, Visit), and footer with address/hours placeholders. Return full files.

Prompt 2: home page

</> Plain text

Build the home page. Sections in order: (1) hero: large heading, one-line subtext, two buttons (Browse collection, Order on WhatsApp), one big photo masked in a circle or lens shape on the right (stacked on mobile); (2) featured frames: 4 products in a grid pulled from products.ts; (3) shop by category: 4 large tiles with photo and label; (4) "Why Mehar for Eyes": three short factual points, no icons from icon packs, use numbers 01 02 03 as typographic markers; (5) visit us: address, hours, embedded map placeholder, Call and WhatsApp buttons. Keep copy specific and short. Do not use any placeholder marketing fluff.

Prompt 3: collection + filters

</> Plain text

Build /collection with client-side filtering by category, gender, and frame shape using URL search params so filtered views are shareable. Product card: image (4:5), name, brand, small material/color line, and an "Order on WhatsApp" text button. Hover swaps to a second image if available. Include an empty state. No pagination yet.

Prompt 4: product page

</> Plain text

Build /collection/[slug] using generateStaticParams from products.ts. Layout: image gallery on left (thumbnails, swipe on mobile), details on right: name, brand, specs table, description, and a primary "Order on WhatsApp" button whose message is prefilled: "Hi Mehar for Eyes, I want to order: {product name} ({product url}). Please confirm availability." Add a sticky bottom bar on mobile with the same button. Add "You may also like" with 4 products from the same category.

Prompt 5: global WhatsApp CTA

</> Plain text

Add a floating WhatsApp button, bottom right, solid circle, simple SVG logo, no pulsing/bouncing animation. On mobile it must not overlap the sticky product bar. Use buildWhatsAppLink with a generic greeting message.

Prompt 6: about + SEO

</> Plain text

```
Build /about with the shop story (leave clearly marked TODO text for me to fill in), address, hours, map, and contact. Then add: per-page metadata, Open Graph tags, LocalBusiness JSON-LD schema (optician) with address/phone/hours placeholders, sitemap.xml, robots.txt, and a favicon. Target local searches like "optician in [city]" and "eyeglasses [city]".
```

Prompt 7: polish pass (run last)

```
</> Plain text

Audit the whole site for AI-generated look. Check: any generic copy, inconsistent spacing, more than one accent color, gradients, emoji, default shadows, icon-pack icons, uneven image aspect ratios, layout shift. Fix everything you find. Then run a Lighthouse-style review for mobile and fix anything under 90 for performance.
```

Tips so Cursor doesn't drift

- One prompt per step, review in the browser before moving on. Chaining all prompts at once gives you the slop.
- If output looks generic, don't say "make it better." Say exactly what's wrong: "heading too small, section padding too tight, remove the card shadows."
- Upload a reference screenshot of sites you like (look at Warby Parker, Cubitts, Oliver Peoples for structure, not to copy) and tell Cursor "match this spacing and typography feel, not the content."

Send me the shop's city and whether you want Urdu support, and I'll tune the SEO and bilingual setup prompts.

2026-10-23 23:26:33 **You**

for now we will use images ffrom google btut here will eb an admin dashboard through where he can upload actual good pictures of frams and glasess and lense etc

Claude

Good plan, but it changes the architecture. `products.ts` is now only a dev stub. Products and images must live in a database + storage, and the site reads from there.

Google images: fine as placeholders while building. Do not launch with them. They're copyrighted, they look inconsistent, and they make the site look like every other cheap optical site. Treat the launch

gate as: father uploads real photos first.

Recommended stack

Next.js + Supabase (Postgres + Auth + Storage in one). Fastest path to a working admin, and ecommerce can be added on the same DB later.

- Public site: reads products from Supabase (server components, cached/revalidated).
- `/admin`: login-protected dashboard.
- Images: Supabase Storage bucket, public read, admin-only write.

You could do this with your Go backend, but it means building auth, upload handling, and image resizing yourself. Not worth it for this project.

Admin must be built for your father, not for you

- Phone-first. He will upload straight from the phone camera.
- Big buttons, minimal fields, English labels now (Urdu labels optional later).
- Auto-resize and compress on upload, so 8MB phone photos don't kill mobile load times.
- One "in stock / out of stock" toggle per product.

Update the Cursor rules first

In `.cursor/rules/mehar.mdc`, replace the products rule with:

```
</> Plain text

- Products come only from the Supabase `products` table via /lib/products.ts data functions. UI never hardcodes product info. /data/products.ts is a dev seed only, to be deleted before launch.
- Images are stored in Supabase Storage bucket `product-images`. Always render through next/image.
- Admin routes live under /app/admin and must be server-side protected. Never trust client checks.
```

Prompts

Prompt A: database + storage

```
</> Plain text

Set up Supabase for this project. Give me: (1) a full SQL migration creating tables: products (id uuid, slug unique, name, brand, category, gender, frame_shape, material, color, price
```

nullable, description, in_stock boolean default true, featured boolean default false, sort_order int, created_at), product_images (id, product_id fk cascade, url, alt, position); (2) a storage bucket `product-images` with public read; (3) RLS policies: public can select products and product_images, only authenticated admin can insert/update/delete, and only admin can write to the bucket. (4) /lib/supabase/server.ts and client.ts using @supabase/ssr. (5) a seed script that inserts my 6 placeholder products. Return full files and the exact commands to run.

Prompt B: swap the site to the DB

</> Plain text

Replace all reads from /data/products.ts with data functions in /lib/products.ts (getProducts with filters, getProductBySlug, getFeatured, getRelated) querying Supabase. Use ISR with revalidate 60 plus on-demand revalidation via revalidatePath after admin changes. Keep generateStaticParams working. Return full changed files.

Prompt C: admin auth

</> Plain text

Add Supabase email/password auth for /admin. Create /admin/login. Protect all /admin routes with middleware AND server-side session checks. Only allow users whose email matches env ADMIN_EMAIL. No public signup route. Logged-out users redirect to /admin/login. Return full files.

Prompt D: product list + form

</> Plain text

Build /admin: a mobile-first product list (thumbnail, name, category, in-stock toggle, featured toggle, edit and delete buttons with confirm dialog). Build /admin/products/new and /admin/products/[id]/edit with one form: name, brand, category (select), gender (select), frame shape (select), material, color, price (optional), description, in stock, featured. Use server actions with zod validation. Auto-generate slug from name, ensure uniqueness. Revalidate public pages after every save. Big touch-friendly inputs, plain labels, clear success and error messages. No UI library.

Prompt E: image upload (the important one)

</> Plain text

Add multi-image upload to the product form. Requirements: accept camera capture and gallery on mobile (input accept="image/*"), client-side resize to max 1600px on the long edge and convert to WebP at ~0.8 quality before upload, show upload progress and thumbnails, drag to reorder (touch friendly), first image is the cover, delete individual images (also remove from storage), require alt text auto-filled from product name. Upload to Supabase Storage

```
under products/{productId}/{uuid}.webp. Limit 8 images per product. Handle failures with retry. Return full files.
```

Prompt F: hardening

```
</> Plain text

Security review of the admin: verify RLS blocks anonymous writes, verify server actions re-check the session, verify file type and size limits server-side, verify no service role key is exposed to the client, add rate limiting on login. Fix everything you find and list what changed.
```

Blunt warnings

- Never put the Supabase **service role key** in client code. Cursor sometimes does this. Check.
- Test the admin on a real phone with your father's actual workflow before showing him. If he can't add a product in under 2 minutes, simplify the form.
- Back up the DB and bucket occasionally. He will eventually delete something by mistake.
- Add a delete confirmation and consider "hide" (in stock off) as the default over deleting.

Tell me if you'd rather use Cloudinary for image handling instead of Supabase Storage. It auto-optimizes and serves faster on weak networks, at the cost of one more service.

2026-10-23 23:28:19 **You**

i am confused, what is supabase, never heard of it, which tech stack we are uszing

Claude

Fair, I dropped it on you without explaining.

What Supabase is

A ready-made backend you rent instead of building. It gives you three things in one place:

1. **Database** (Postgres): stores products, names, prices, categories.
2. **Auth**: login system for your father's admin dashboard.
3. **Storage**: where uploaded photos live (like a folder on the internet with a URL for each image).

Think of it as "the Go backend + database + file upload server" that you'd otherwise write and host yourself. It has a free tier that is more than enough for this site.

The full stack

Part	Tech	Job
Website + admin UI	Next.js + TypeScript + Tailwind	Public pages and `/admin` pages, one project
Data + login + images	Supabase	Products table, admin login, photo storage
Hosting	Vercel	Deploys the Next.js app from GitHub
Orders	WhatsApp `wa.me` links	No backend needed for orders

Flow: father logs into `/admin` → uploads photos and product details → saved in Supabase → public site reads from Supabase and shows them.

Your other option (uses your Go skills)

- Next.js frontend (same)
- Go API + Postgres on a VPS
- Images on S3 or Cloudinary
- You write auth, upload endpoints, image resizing, deployment, backups

This is more work, more to maintain, more that can break. Choose it only if you want full control or already have a VPS you want to use.

My recommendation

Supabase. Reason: this is a showcase site, and your time is better spent on design and the admin UX than on rebuilding auth and file uploads. If ecommerce comes later, Supabase's Postgres still works; you can add a Go service next to it if needed.

Tell me which one, and I'll give you the exact Cursor prompts for that path. If Supabase, the prompts A-F from my last message are ready to use as written. If Go, I'll rewrite them.

2026-10-23 23:28:45 You

so i have to buy supabase?

Claude

No. Supabase has a free plan, and for this project it is enough to start.

The free tier gives you 500 MB of database storage, 50,000 monthly active users, 5 GB of egress, 500 MB of file storage, and up to 2 active projects. One source lists 1 GB of file storage instead, so check supabase.com/pricing when you sign up. Either way, it covers a catalog of a few hundred products with compressed photos. That's why prompt E resizes images to WebP before upload.

The catch you need to know

Free projects auto-pause after one week of inactivity. If nobody opens the site or admin for 7 days, the database goes to sleep and the public site shows errors or empty pages until you wake it manually in the dashboard. For a shop's public site, that is a real problem.

Fixes:

1. **Free workaround:** a scheduled ping (Vercel cron or a GitHub Action) that hits a cheap query once a day. This keeps it active. It works, but it's a hack.
2. **Proper fix:** the Pro plan. It costs $25/month. Pro projects stay running. Upgrade when the site goes live and real customers use it.

Cost summary for this project

Part	Cost
Supabase (dev + early launch)	$0

Part	Cost
Vercel hosting (Hobby plan)	$0 (Vercel's terms restrict Hobby to non-commercial use, so for a business site you'll likely need Pro at $20/mo; check their pricing page)
Domain (.com or .pk)	~$10-15/year
WhatsApp links	$0

If you want zero monthly cost and full control, the Go + Postgres on your own VPS route avoids Supabase fees, but you pay with your time and maintenance. For a business site where your father depends on it, I'd still take Supabase Free while building, then pay for Pro at launch.

Next step: create a free account at supabase.com, make a project, and run prompt A from before. Tell me when you hit an error and paste it verbatim.

2026-10-23 23:29:26 **You**

why cant i just make my own backend? i am a devloper also not a noob

Claude

You can. I wasn't saying you couldn't. I picked Supabase to save you time, not because the backend is beyond you. For a Go developer, this is a legitimate and arguably better fit.

Honest tradeoffs

What you gain:

- $0 extra if you already have a VPS. No auto-pause, no vendor limits.
- You own the data and the images.
- Ecommerce later (orders, stock, payments like JazzCash/Easypaisa) is natural to extend in Go.
- It matches your strongest skill.

What you take on:

- Auth, upload handling, image processing, backups, deploys, and uptime are all yours.
- If the site goes down, your father calls you. Permanently.
- Security mistakes are on you (file upload endpoints are a classic attack surface).

None of it is hard for you. It's maintenance, not difficulty. If you accept being the sysadmin for this site, go Go.

Stack

Part	Tech
Public site + admin UI	Next.js + TypeScript + Tailwind
API	Go (chi router) + pgx + sqlc
DB	Postgres
Migrations	golang-migrate
Images	Stored on VPS disk (or Cloudflare R2), served via Cloudflare CDN
Auth	Single admin, argon2 password hash, httpOnly session cookie
Deploy	Docker Compose + Caddy (auto HTTPS)

Put **Cloudflare** (free plan) in front of the domain. It caches images at edge nodes, which helps on weak Pakistani mobile connections.

Prompts

Update `.cursor/rules/mehar.mdc`, replace the data rule with:

```
</> Plain text
```

```
- Backend is a separate Go service in /backend (chi, pgx, sqlc, golang-migrate, Postgres). Frontend never talks to the DB directly, only to the Go API.
- Public pages fetch from the API server-side in Next.js with revalidation. Admin pages call the API with the session cookie.
- Images are stored on disk under /data/uploads and served by Caddy at /uploads/*. DB stores only the relative path.
- Go rules: no global state, context passed everywhere, errors wrapped, all inputs validated, parameterized queries only via sqlc. Give full files, not snippets.
```

Prompt A: backend foundation + schema

```
</> Plain text

Create /backend as a Go 1.22+ module. Structure: cmd/api/main.go, internal/{config,db,http,auth,products,images}. Use chi, pgx/v5, sqlc, golang-migrate. Write migrations for: products (id uuid pk, slug unique, name, brand, category, gender, frame_shape, material, color, price int nullable, description, in_stock bool default true, featured bool default false, sort_order int, created_at, updated_at), product_images (id uuid, product_id fk on delete cascade, path, alt, position int), admin_sessions (id, token_hash, expires_at). Write sqlc queries and config via env vars. Add docker-compose.yml with postgres and the api. Add /healthz. Return all files and exact commands to run.
```

Prompt B: admin auth

```
</> Plain text

Implement single-admin auth in Go. Admin email and argon2id password hash come from env (provide a small cmd/hashpw tool to generate the hash). POST /api/admin/login sets an httpOnly, Secure, SameSite=Lax session cookie with a random 32-byte token (store only its SHA-256 in admin_sessions, 7 day expiry). POST /api/admin/logout. Middleware protecting all /api/admin/* routes. Add login rate limiting per IP (5 attempts per 15 minutes) and constant-time comparisons. CSRF protection via a double-submit token or Origin header check on mutating requests. Return full files.
```

Prompt C: product API

```
</> Plain text

Build the products API. Public: GET /api/products (filters: category, gender, frame_shape, featured, q; pagination), GET /api/products/{slug}, GET /api/products/{slug}/related. Admin: POST /api/admin/products, PUT /api/admin/products/{id}, DELETE /api/admin/products/{id}, PATCH /api/admin/products/{id}/stock, PATCH /api/admin/products/{id}/featured. Validate every field, generate unique slug from name, return consistent JSON errors. Add tests for slug generation and validation. Return full files.
```

Prompt D: image upload (security-critical)

```
</> Plain text

Build image upload in Go: POST /api/admin/products/{id}/images (multipart, up to 8 files, max 5MB each, max 8 images per product). Server-side: sniff real content type with http.DetectContentType (allow only jpeg, png, webp, never trust extension or header), decode and re-encode to strip EXIF and any embedded payload, resize to max 1600px, also generate a 600px thumbnail, save as WebP using libvips via bimg (or kolesa-team/go-webp), name files with uuid, write to /data/uploads/products/{id}/. PUT to reorder images, DELETE to remove (delete files from disk too). Limit request body with http.MaxBytesReader. Return full files and the Dockerfile changes needed for libvips.
```

Prompt E: frontend

```
</> Plain text

Build the Next.js admin: /admin/login, /admin product list (mobile-first, thumbnail, in-stock toggle, featured toggle, edit, delete with confirm), /admin/products/new and /[id]/edit form with all product fields and multi-image upload (camera and gallery input, client-side resize to WebP before upload, progress, touch reorder, delete). Call the Go API with credentials: 'include'. Protect admin routes in middleware by checking a session probe endpoint. Then switch all public pages to fetch from the Go API server-side with revalidate 60. Remove /data/products.ts. Return full files.
```

Prompt F: deploy and backups

```
</> Plain text

Write production deployment: docker-compose.prod.yml with caddy, next app, go api, postgres, a volume for /data/uploads. Caddyfile with auto HTTPS, reverse proxy /api to Go, serve /uploads/* statically with long cache headers, security headers. A nightly backup script: pg_dump plus rsync of /data/uploads to a second location, keep 14 days, with cron and restore instructions. Add a GitHub Actions workflow to build and deploy over SSH. Return full files and exact commands.
```

Warnings

- **Upload endpoint:** the re-encode step in prompt D is not optional. It's what kills malicious files disguised as images.
- **Backups:** test a restore once. An untested backup is not a backup.
- **Cloudflare:** set cache rules for `/uploads/*`. Do not cache `/api/admin/*`.
- **Don't expose Postgres** to the public internet. Docker internal network only.

Where are you hosting? If you already have a VPS at Tawha or elsewhere, tell me the OS and RAM, and I'll adjust the Docker and Caddy setup.

2026-10-23 23:30:24 **You**

all the prompts u gave me i actually neer gave them to cursor so which is the first prompt to giv it

Claude

Ignore the Supabase prompts, since you're going with your own Go backend. Here's the actual order.

Order

1. **Rules file** (below), once
2. **Original Prompt 1** (frontend foundation, below)
3. **Original Prompts 2, 3, 4, 5** (home, collection, product page, WhatsApp button), using the dev stub `products.ts` with placeholder images
4. **Go backend Prompts A, B, C, D**
5. **Go backend Prompt E** (admin UI, swaps the site from `products.ts` to the API)
6. **Original Prompts 6 and 7** (about/SEO, polish)
7. **Go backend Prompt F** (deploy and backups)

Reason: you get something visible in the browser in step 3 and can judge the design before building the backend. One prompt at a time, check it in the browser, then move on.

Step 1: create `.cursor/rules/mehar.mdc`

Make the file, set it to "Always apply", and paste:

```
</> Plain text

Project: Mehar for Eyes, a showcase website for an established optical business in Pakistan.
Not ecommerce yet. All orders happen via WhatsApp. Ecommerce will be added later, so keep
product data separate from UI.

Stack: Next.js App Router, TypeScript, Tailwind CSS, next/image. No UI component libraries
unless I ask. Backend will be a separate Go service in /backend (chi, pgx, sqlc, golang-
migrate, Postgres), built later.

Design rules:
- Warm off-white background (#FAF7F2), near-black text (#1A1A1A), one accent color (#5A2E1F).
No gradients, no glassmorphism, no purple/blue.
```

```
- Fonts via next/font: Fraunces (headings), DM Sans (body).
- Generous whitespace, large product imagery, hairline borders (1px), subtle motion only (fade/translate on scroll, 200-300ms).
- No emojis in UI. No lorem ipsum. No generic marketing copy like "Welcome to" or "Discover the best". Write short, specific, human copy.
- Mobile first. Must look correct at 360px width.
- Accessible: alt text on all images, visible focus states, contrast AA.
- Performance: all images through next/image with sizes, lazy load below the fold.

Data rules:
- For now, products come only from /data/products.ts, a dev stub. UI never hardcodes product info. All data access goes through functions in /lib/products.ts so the source can be swapped to the Go API later without touching components.
- Placeholder images live in /public/placeholders. Never hotlink external image URLs.
- WhatsApp link logic lives only in /lib/whatsapp.ts.

Code rules:
- Give full files, not snippets.
```

Step 2: first prompt to give Cursor

</> Plain text

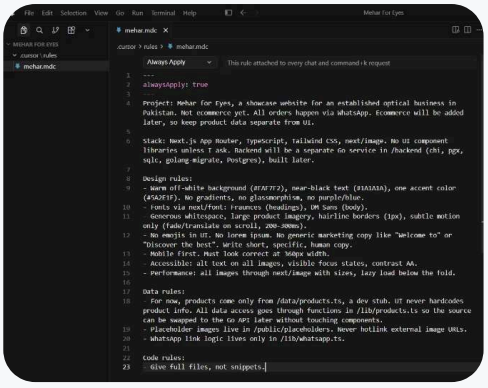
```
Set up the project structure: app/ with routes /, /collection, /collection/[slug], /about. Create /data/products.ts with a typed Product (id, slug, name, brand, category, gender, frameShape, material, color, price optional, images[], description, inStock, featured). Add 6 placeholder products clearly marked TODO so I can replace them, with image paths pointing to /public/placeholders/ (create that folder and a README saying to drop downloaded placeholder photos there). Create /lib/products.ts exporting async functions getProducts(filters), getProductBySlug(slug), getFeatured(), getRelated(slug) that read from /data/products.ts for now. Create /lib/whatsapp.ts exporting buildWhatsAppLink(message) using wa.me with the number from env NEXT_PUBLIC_WHATSAPP_NUMBER (format 92XXXXXXXXXX, no + or leading 0) and encodeURIComponent. Add .env.example. Add the root layout with next/font (Fraunces and DM Sans), header (text logo "Mehar for Eyes", nav: Collection, About, Visit), and footer with address and hours placeholders marked TODO. Set up Tailwind theme tokens for the colors in the rules. Return full files and the exact commands to run the project.
```

Two things before you run it:

- Download 6 to 8 placeholder photos into `/public/placeholders` yourself. Cursor can't fetch Google images, and hotlinking breaks.
- Put your real WhatsApp number in `.env.local` as `NEXT_PUBLIC_WHATSAPP_NUMBER=92XXXXXXXXXX`.

Run it, open `localhost:3000`, and paste me any error verbatim. When it works, tell me and I'll give you the next prompt cleaned up for exactly where you are.

2026-10-23 23:33:33 **You**



liek this?

Claude

Yes, that's correct. `alwaysApply: true` plus the "Always Apply" dropdown means it attaches to every chat.

One gap: your folder is empty apart from `.cursor`, so there's no Next.js project yet. Scaffold it first, in the VS Code/Cursor terminal at the project root:

```
</> Plain text

npx create-next-app@latest . --typescript --tailwind --eslint --app --no-src-dir --import-alias "@/*"
```

If it asks questions you didn't cover with flags (Turbopack, React Compiler, etc.), take the defaults. If it refuses because the folder isn't empty, tell me the exact message.

Then verify it runs:

```
</> Plain text

npm run dev
```

Open `http://localhost:3000`. If you see the default Next.js page, you're good.

Then open Cursor chat in **Agent mode** and paste the first prompt:

```
</> Plain text
```

```
Set up the project structure: app/ with routes /, /collection, /collection/[slug], /about. Create /data/products.ts with a typed Product (id, slug, name, brand, category, gender, frameShape, material, color, price optional, images[], description, inStock, featured). Add 6 placeholder products clearly marked TODO so I can replace them, with image paths pointing to /public/placeholders/ (create that folder and a README saying to drop downloaded placeholder photos there). Create /lib/products.ts exporting async functions getProducts(filters), getProductBySlug(slug), getFeatured(), getRelated(slug) that read from /data/products.ts for now. Create /lib/whatsapp.ts exporting buildWhatsAppLink(message) using wa.me with the number from env NEXT_PUBLIC_WHATSAPP_NUMBER (format 92XXXXXXXXXX, no + or leading 0) and encodeURIComponent. Add .env.example. Update the root layout with next/font (Fraunces and DM Sans), header (text logo "Mehar for Eyes", nav: Collection, About, Visit), and footer with address and hours placeholders marked TODO. Set up the color tokens from the rules in the Tailwind theme (this project uses the Tailwind version created by create-next-app, so use whatever config style that version uses). Return full files and the exact commands to run the project.
```

Before you check the result: put your WhatsApp number in `.env.local` as `NEXT_PUBLIC_WHATSAPP_NUMBER=92XXXXXXXXXX`, and drop a few placeholder photos in `public/placeholders/`.

Paste me any error verbatim, or a screenshot once it renders.

2026-10-23 23:34:20 **You**

explain in easy words bruh i literally couldnt understand

Claude

Your rules file is correct. Now do these 4 steps, in order.

Step 1: make the website project

Open the terminal in Cursor (Terminal menu, then New Terminal). Paste this and press Enter:

```
</> Plain text

npx create-next-app@latest . --typescript --tailwind --eslint --app --no-src-dir --import-alias "@/*"
```

This creates the empty website inside your folder. If it asks questions, just press Enter on each one.

Step 2: run it

```
Plain text

npm run dev
```

Open `http://localhost:3000` in your browser. If you see a Next.js welcome page, it works.

Step 3: put your WhatsApp number in a file

Create a file called `.env.local` in the main folder. Write this inside, with your real number (start with 92, no + and no 0):

```
Plain text

NEXT_PUBLIC_WHATSAPP_NUMBER=92XXXXXXXXXX
```

Step 4: give Cursor the first command

Open the Cursor chat, switch it to **Agent mode**, and paste this:

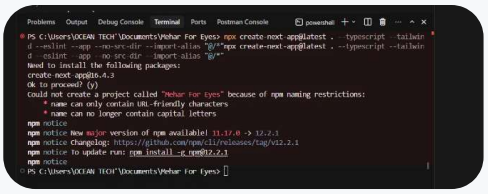
```
Plain text

Set up the project structure: app/ with routes /, /collection, /collection/[slug], /about. Create /data/products.ts with a typed Product (id, slug, name, brand, category, gender, frameShape, material, color, price optional, images[], description, inStock, featured). Add 6 placeholder products clearly marked TODO so I can replace them, with image paths pointing to /public/placeholders/ (create that folder and a README saying to drop downloaded placeholder photos there). Create /lib/products.ts exporting async functions getProducts(filters), getProductBySlug(slug), getFeatured(), getRelated(slug) that read from /data/products.ts for now. Create /lib/whatsapp.ts exporting buildWhatsAppLink(message) using wa.me with the number from env NEXT_PUBLIC_WHATSAPP_NUMBER (format 92XXXXXXXXXX, no + or leading 0) and encodeURIComponent. Add .env.example. Update the root layout with next/font (Fraunces and DM Sans), header (text logo "Mehar for Eyes", nav: Collection, About, Visit), and footer with address and hours placeholders marked TODO. Set up the color tokens from the rules in the Tailwind theme (this project uses the Tailwind version created by create-next-app, so use whatever config style that version uses). Return full files and the exact commands to run the project.
```

Later, download 6 to 8 glasses photos from Google and put them in the `public/placeholders` folder (Cursor creates it). Do this after Step 4.

If any step gives an error, copy the error text and send it to me.

2026-10-23 23:35:53 **You**



Claude

Two small problems, both easy.

Problem 1: the command got pasted twice (you can see it repeated in the terminal). Paste it only once.

Problem 2: your folder is called "Mehar For Eyes". npm doesn't allow capital letters or spaces in project names, so using `.` (the current folder) fails.

Fix: make the project in a new folder with a proper name, inside your current folder.

Step 1. Paste this once and press Enter:

```
</> Plain text

npx create-next-app@latest mehar-for-eyes --typescript --tailwind --eslint --app --no-src-dir
--import-alias "@/*"
```

If it asks questions, press Enter on each one.

Step 2. Move your rules folder into the new project:

```
</> Plain text

Move-Item .cursor mehar-for-eyes\
```

Step 3. Open the new folder in Cursor: File menu, Open Folder, then pick `Mehar For Eyes\mehar-for-eyes`. Cursor will reload. You should see `.cursor`, `app`, `package.json` etc. in the left panel.

Step 4. Open a new terminal (Terminal menu, New Terminal) and run:

```
</> Plain text

npm run dev
```

Open `http://localhost:3000`. If you see the Next.js page, tell me and do the rest (the `.env.local` file and the Cursor Agent prompt) from my last message. Same instructions, just inside this new folder.

If anything errors, send a screenshot again.

2026-10-23 23:43:01 **You**

se this:
PS C:\Users\OCEAN TECH'\Documents\Mehar For Eyes> npx create-next-app@latest mehar-for-eyes --typescript --tailwind --eslint --app --no-src-dir --import-alias "@/*"

Using defaults for unprovided options:

--ts TypeScript (use --js for JavaScript)
--no-react-compiler No React Compiler (use --react-compiler for React Compiler)
--cache-components Cache Components (use --no-cache-components for No Cache Components)
Creating a new Next.js app in C:\Users\OCEAN TECH'\Documents\Mehar For Eyes\mehar-for-eyes.

Using npm.

Initializing project with template: app-tw

Installing dependencies:
- next
- react
- react-dom

Installing devDependencies:
- @tailwindcss/turbopack
- @types/node
- @types/react
- @types/react-dom
- eslint
- eslint-config-next
- tailwindcss
- typescript

npm warn deprecated eslint@9.39.5: This version is no longer

supported. Please see https://eslint.org/version-support for other options.

added 357 packages, and audited 358 packages in 4m

145 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
npm warn allow-scripts 1 package has install scripts not yet covered by allowScripts:
npm warn allow-scripts   unrs-resolver@1.12.2 (postinstall: node postinstall.js)
npm warn allow-scripts
npm warn allow-scripts Run `npm approve-scripts --allow-scripts-pending` to review, or `npm approve-scripts <pkg>` to allow.

Generating route types...
✓ Types generated successfully

Initialized a git repository.

Success! Created mehar-for-eyes at C:\Users\OCEAN TECH'\Documents\Mehar For Eyes\mehar-for-eyes

PS C:\Users\OCEAN TECH'\Documents\Mehar For Eyes>

Claude

It worked. "Success! Created mehar-for-eyes" is the line that matters. The yellow `warn` lines are harmless, ignore them.

Now do these in order.

Step 1. Move your rules folder into the new project. In the same terminal, paste:

```
</> Plain text

Move-Item .cursor mehar-for-eyes\
```

Step 2. Open the new folder in Cursor: File menu, Open Folder, then choose `Documents\Mehar For Eyes\mehar-for-eyes`. Cursor reloads. In the left panel you should see `.cursor`, `app`, `package.json`.

Step 3. Open a new terminal there (Terminal menu, New Terminal) and run:

```
</> Plain text

npm run dev
```

Open `http://localhost:3000` in your browser. You should see the default Next.js page.

Step 4. Create a file called `.env.local` in the main folder (next to `package.json`). Put this inside, with your real number (starts with 92, no + and no 0):

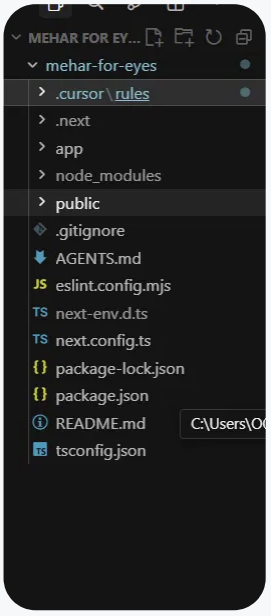
```
</> Plain text

NEXT_PUBLIC_WHATSAPP_NUMBER=92XXXXXXXXXX
```

Step 5. Open the Cursor chat, switch to **Agent mode**, and paste the first prompt from my earlier message (the long one starting with "Set up the project structure...").

Tell me when the localhost page shows up, or screenshot any error.

2026-10-23 23:43:47 **You**



liek this

Claude

Yes, exactly like this. Your folder is correct: `.cursor\rules`, `app`, `public`, `package.json` are all there. The `.next` folder means the dev server already ran once, which is good.

Two things are still missing:

1. Create `.env.local`. It's not in your list yet. Right-click an empty area in the left panel, choose New File, name it `.env.local`, and put this inside (your real number, starts with 92, no + and no 0):

```
</> Plain text

NEXT_PUBLIC_WHATSAPP_NUMBER=92XXXXXXXXXX
```

2. Check the site runs. Open a terminal (Terminal menu, New Terminal) and run:

```
</> Plain text

npm run dev
```

Open `http://localhost:3000`. If you see the Next.js page, it works. Leave this terminal running.

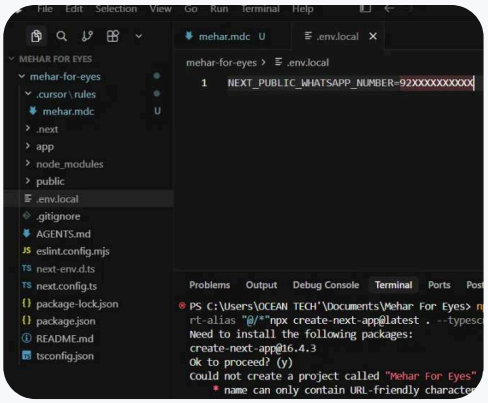
3. Give Cursor the first command. Open the Cursor chat, switch to **Agent mode**, and paste this:

```
</> Plain text

Set up the project structure: app/ with routes /, /collection, /collection/[slug], /about.
Create /data/products.ts with a typed Product (id, slug, name, brand, category, gender,
frameShape, material, color, price optional, images[], description, inStock, featured). Add 6
placeholder products clearly marked TODO so I can replace them, with image paths pointing to
/public/placeholders/ (create that folder and a README saying to drop downloaded placeholder
photos there). Create /lib/products.ts exporting async functions getProducts(filters),
getProductBySlug(slug), getFeatured(), getRelated(slug) that read from /data/products.ts for
now. Create /lib/whatsapp.ts exporting buildWhatsAppLink(message) using wa.me with the number
from env NEXT_PUBLIC_WHATSAPP_NUMBER (format 92XXXXXXXXXX, no + or leading 0) and
encodeURIComponent. Add .env.example. Update the root layout with next/font (Fraunces and DM
Sans), header (text logo "Mehar for Eyes", nav: Collection, About, Visit), and footer with
address and hours placeholders marked TODO. Set up the color tokens from the rules in the
Tailwind theme (this project uses the Tailwind version created by create-next-app, so use
whatever config style that version uses). Return full files and the exact commands to run the
project.
```

When Cursor finishes, send me a screenshot of the browser or any error.

2026-10-23 23:44:59 **You**



done

Claude

The file is in the right place. Two small things before the next step.

1. Put your real number. Right now it says `92XXXXXXXXXX`. Replace the X's with the actual WhatsApp number of the shop. Example: if the number is 0300 1234567, write:

```
Plain text

NEXT_PUBLIC_WHATSAPP_NUMBER=923001234567
```

Drop the first 0, start with 92, no spaces, no +. Then press Ctrl+S to save.

2. Ignore the red text in the terminal. That is the old error from before. Open a fresh terminal: Terminal menu, then New Terminal. Make sure the path shows `...\Mehar For Eyes\mehar-for-eyes>` at the end, then run:

```
Plain text

npm run dev
```

Open `http://localhost:3000` in your browser. You should see the default Next.js page.

3. Then give Cursor the first command. Open the Cursor chat, switch it to **Agent mode**, and paste the long prompt from my earlier message (the one starting with "Set up the project structure...").

Send me a screenshot of the browser when it loads, or of any error.

2026-10-23 23:46:45 **You**

+92 321 3763432

this

Claude

That's the number, so the line is:

```
Plain text

NEXT_PUBLIC_WHATSAPP_NUMBER=923213763432
```

Replace the whole line in `.env.local` with that, then press Ctrl+S to save.

Next steps:

1. Open a fresh terminal (Terminal menu, New Terminal), check the path ends with `...\mehar-for-eyes>`, and run:

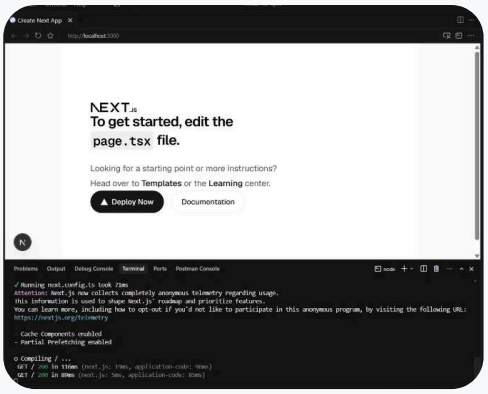
```
Plain text

npm run dev
```

2. Open `http://localhost:3000` and check the Next.js page loads.
3. Open the Cursor chat, switch to **Agent mode**, and paste the long first prompt ("Set up the project structure...").

Send a screenshot when Cursor finishes or if anything errors.

2026-10-23 23:48:24 **You**



Claude

The site runs. That Next.js page on localhost:3000 is exactly what you should see at this point.

Now give Cursor the first command.

1. Keep the `npm run dev` terminal running. Don't close it.
2. Open the Cursor chat panel (the chat icon on the right side, or press Ctrl+L).
3. Switch it to **Agent mode** using the dropdown at the bottom of the chat box.
4. Paste this and press Enter:

```
</> Plain text

Set up the project structure: app/ with routes /, /collection, /collection/[slug], /about. Create /data/products.ts with a typed Product (id, slug, name, brand, category, gender, frameShape, material, color, price optional, images[], description, inStock, featured). Add 6 placeholder products clearly marked TODO so I can replace them, with image paths pointing to /public/placeholders/ (create that folder and a README saying to drop downloaded placeholder photos there). Create /lib/products.ts exporting async functions getProducts(filters), getProductBySlug(slug), getFeatured(), getRelated(slug) that read from /data/products.ts for now. Create /lib/whatsapp.ts exporting buildWhatsAppLink(message) using wa.me with the number from env NEXT_PUBLIC_WHATSAPP_NUMBER (format 92XXXXXXXXXX, no + or leading 0) and encodeURIComponent. Add .env.example. Update the root layout with next/font (Fraunces and DM Sans), header (text logo "Mehar for Eyes", nav: Collection, About, Visit), and footer with address and hours placeholders marked TODO. Set up the color tokens from the rules in the Tailwind theme (this project uses the Tailwind version created by create-next-app, so use whatever config style that version uses). This project is Next.js 16 with Cache Components enabled, so follow the Next.js 16 conventions in AGENTS.md and node_modules/next/dist/docs if present. Return full files and the exact commands to run the project.
```

I added one line about Next.js 16, because your project has "Cache Components enabled" (visible in your terminal). That's a newer setting that can cause strange errors if Cursor writes older-style code.

5. Let Cursor work. It will create files and may ask you to click "Accept" or "Keep". Accept them.

6. When it finishes, refresh `localhost:3000` and send me a screenshot. If a red error shows up in the browser or terminal, screenshot that instead.

Also, download 6 to 8 glasses photos from Google into `public/placeholders` after Cursor creates that folder.

2026-10-23 23:58:50 **You**

ok done it built it whats next

Claude

Next is the home page. Before you paste it:

1. Make sure the photos are in `public/placeholders` (6 to 8 glasses photos you downloaded). Without them the page will show broken images.
2. Refresh `localhost:3000`. If you see a red error, screenshot it and send it to me first, before anything else.

If it loads fine, paste this into Cursor chat (Agent mode):

```
</> Plain text

Build the home page at app/page.tsx. Use products from lib/products.ts only, never hardcode
product info. Sections in order:

1. Hero: large heading (Fraunces), one short line of text, two buttons (Browse collection
linking to /collection, and Order on WhatsApp using buildWhatsAppLink with a generic
greeting). One large photo on the right masked in a circle (lens shape), stacked below the
text on mobile. Use the first featured product image.

2. Featured frames: 4 products from getFeatured() in a grid (2 columns on mobile, 4 on
desktop). Each card: image 4:5 ratio, name, brand, link to /collection/[slug]. Hairline
border, no shadows.

3. Shop by category: 4 large tiles (Men, Women, Kids, Sunglasses) each with a photo from a
product in that category and a label, linking to /collection?category=...

4. Why Mehar for Eyes: three short factual points marked 01, 02, 03 in Fraunces as
typographic markers, no icon packs. Write the copy as TODO placeholders that I will replace
with real facts about the shop.

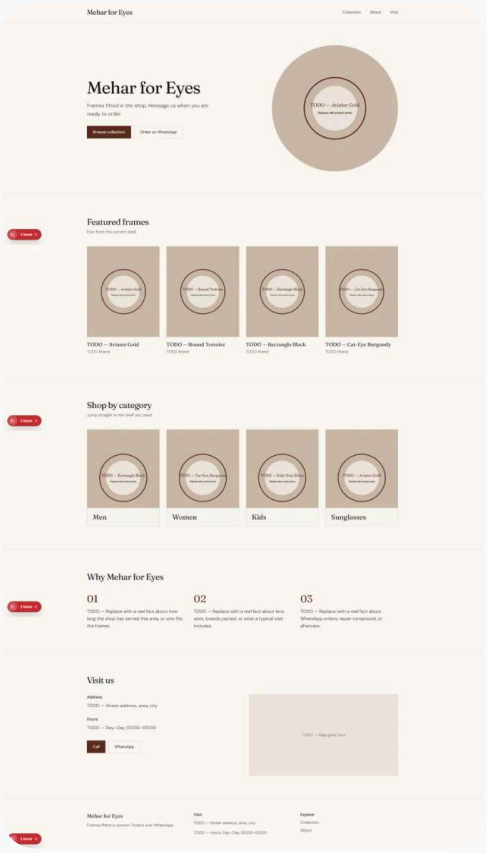
5. Visit us: address and hours (TODO placeholders), Call button (tel: link) and WhatsApp
```

```
button, and a box where a map will go later.

Rules: follow the design rules in .cursor/rules. No generic marketing copy. Mobile first,
check 360px width. Subtle fade-in on scroll only. Return full files.
```

When Cursor finishes, send me a screenshot of the home page (top part and scrolled down).

2026-10-24 00:10:09 **You**



here it is

Claude

Good start. The layout, colors and fonts follow the rules, with no gradients or generic marketing text. The structure is what we want.

Fix first: the red "1 Issue" badge

That red badge on the left is Next.js telling you there's an error or warning. Click it, read the message, and send it to me (screenshot is fine). If you want to skip waiting, paste this into Cursor:

```
</> Plain text
```

```
The Next.js dev overlay shows "1 Issue" on the home page. Open the dev server terminal output and the browser console, find the exact error or warning, explain in one sentence what causes it, and fix it. Return full changed files.
```

Don't move on until the badge is gone.

What's still wrong

1. **No real photos yet.** The beige circles are Cursor's placeholder boxes. Put your downloaded glasses photos in `public/placeholders/`, then tell Cursor:

```
</> Plain text

I added photos to public/placeholders: [paste the file names]. Update data/products.ts so each product uses one of these files as its main image, and a second one as its hover image where available. Keep the TODO markers on names and brands. Return full files.
```

2. **The hero repeats the shop name.** The logo at the top already says "Mehar for Eyes", and the big heading says it again. Later, when you have the real shop story, change the heading to something specific, like what the shop is known for.

3. **No WhatsApp button in the header.** On mobile, people need it always visible. We'll add the floating button in Prompt 5.

Next prompt (after the badge is gone and photos are in): the collection page

```
</> Plain text

Build /collection at app/collection/page.tsx with filtering by category, gender, and frame shape using URL search params, so filtered pages can be shared and the category tiles on the home page work. Use getProducts(filters) from lib/products.ts. Filters: a row of simple text buttons on desktop, and a "Filter" button that opens a slide-up panel on mobile. Show the count of results. Product card: image (4:5 ratio), name, brand, a small material and color line, and an "Order on WhatsApp" text link that uses buildWhatsAppLink with a prefilled message including the product name. If a second image exists, show it on hover. Add an empty state ("No frames match this filter") with a clear-filters link. No pagination yet. Follow the design rules in .cursor/rules. Mobile first, check 360px. Return full files.
```

Send me the "1 Issue" message first, or a screenshot of the collection page once it's built.

2026-10-24 11:08:10 **You**

i will add the photos later through the admin protal so i dont wanna do that now

Claude

Good call. The beige placeholder boxes are fine for now. Skip the photo step.

Do these two things, in order.

1. Fix the red "1 Issue" badge first. Click it, read the message, and send it to me. Or paste this into Cursor (Agent mode) and let it find the problem:

```
</> Plain text

The Next.js dev overlay shows "1 Issue" on the home page. Open the dev server terminal output and the browser console, find the exact error or warning, explain in one sentence what causes it, and fix it. Return full changed files.
```

Refresh the page after. Don't go to step 2 until the badge is gone, or the error will follow you into every new page.

2. Build the collection page. Paste this into Cursor:

```
</> Plain text

Build /collection at app/collection/page.tsx with filtering by category, gender, and frame shape using URL search params, so filtered pages can be shared and the category tiles on the home page work. Use getProducts(filters) from lib/products.ts. Filters: a row of simple text buttons on desktop, and a "Filter" button that opens a slide-up panel on mobile. Show the count of results. Product card: image (4:5 ratio), name, brand, a small material and color line, and an "Order on WhatsApp" text link that uses buildWhatsAppLink with a prefilled message including the product name. If a second image exists, show it on hover. Add an empty state ("No frames match this filter") with a clear-filters link. No pagination yet. Follow the design rules in .cursor/rules. Mobile first, check 360px. Return full files.
```

Then click the Men, Women, Kids, Sunglasses tiles on the home page and check that the collection page filters correctly. Send me a screenshot of the collection page, or the error message if anything breaks.

2026-10-24 11:20:49 You

it buitl it whats next

Claude

Next is the product page, the one that opens when someone clicks a frame.

First, quick check on the collection page:

- Click the Men, Women, Kids, Sunglasses tiles on the home page. Does the collection page filter correctly?
- Is the red "1 Issue" badge gone? If not, send me the message.

If those are fine, paste this into Cursor (Agent mode):

```
</> Plain text

Build the product detail page at app/collection/[slug]/page.tsx using getProductBySlug(slug) from lib/products.ts, and generateStaticParams from the products list. If the slug does not exist, show the not-found page. Follow the Next.js 16 conventions already used in this project.

Layout:
- Desktop: image gallery on the left, details on the right. Mobile: gallery on top, details below.
- Gallery: one large image (4:5 ratio) with thumbnails below it. Clicking a thumbnail swaps the large image. Swipe works on mobile. If only one image exists, show no thumbnails.
- Details: product name (Fraunces), brand, a specs table (category, gender, frame shape, material, color), description, and a stock label ("In stock" or "Out of stock").
- Primary button: "Order on WhatsApp", using buildWhatsAppLink with this message: "Hi Mehar for Eyes, I want to order: {product name} {product url}. Please confirm availability." Build the product url from env NEXT_PUBLIC_SITE_URL plus /collection/{slug}. Add NEXT_PUBLIC_SITE_URL=http://localhost:3000 to .env.example.
- If the product is out of stock, change the button text to "Ask about availability on WhatsApp" with an adjusted message.
- On mobile, add a sticky bottom bar with the same WhatsApp button. Add bottom padding to the page so the bar never covers content.
- Below the details: "You may also like", 4 products from getRelated(slug), same card style as the collection page.
- Add generateMetadata with the product name and description as the page title and description.

Rules: follow the design rules in .cursor/rules. No shadows, no gradients. Mobile first, check 360px. Return full files.
```

Then also:

1. Add this line to your `.env.local` file: `NEXT_PUBLIC_SITE_URL=http://localhost:3000`. When the site goes live, you change it to the real domain.
2. Restart the dev server (Ctrl+C in the terminal, then `npm run dev`) because `.env.local` changes need a restart.

To test: open the collection page, click a product, check the gallery, click the WhatsApp button, and check that WhatsApp opens with the message filled in. Then try a fake link like `localhost:3000/collection/abc` and check it shows a not-found page, not a crash.

Send me a screenshot of the product page (desktop, and mobile view if you can: right-click, Inspect, then toggle the device toolbar). After this comes the floating WhatsApp button, then the About page and SEO.

2026-11-01 21:26:01 **You**

if i port forward i the website cna u check it urslef

Claude

Not really, and I wouldn't do it anyway.

Why I can't check it properly

- My terminal can only reach a short list of approved sites, so I can't open your IP or port.
- I do have a page-fetch tool that can open a public URL you give me, but it only returns the page's text and HTML. It doesn't show me how the page looks: layout, spacing, colors, mobile view. That's the thing I'd be checking, so screenshots from you work better.

Why port forwarding is a bad idea

- It opens a hole from the internet into your PC. The Next.js dev server is not built to face the public, so don't expose it.
- It breaks whenever your IP changes or the PC sleeps.

Better options

1. **Screenshots (easiest).** Send me a screenshot of each page, desktop and mobile. This is all I need to give design feedback.

2. **Vercel preview (best for sharing).** Push the project to GitHub, import it on vercel.com, and you get a free public link. Your father or friends can open it on their phones, and you can send me the link too. Remember to add `NEXT_PUBLIC_WHATSAPP_NUMBER` and `NEXT_PUBLIC_SITE_URL` in Vercel's environment settings.
3. **Temporary tunnel.** If you only want a quick link to test on your own phone, Cloudflare Tunnel or ngrok gives a temporary public URL without opening your router. Close it when you're done.

For now, do option 1. Send me the product page screenshots when it's built, and we'll move on to the floating WhatsApp button.

2026-11-01 21:26:57 **You**

ok so what was the last thing we implemnted cause i dont remebr that , and i also dont remember that i even implmented or not

Claude

Here's where you stand, based on what you told me and showed me.

Confirmed done

1. Rules file `.cursor/rules/mehar.mdc`
2. Next.js project created (`mehar-for-eyes` folder)
3. `.env.local` with the WhatsApp number (923213763432)
4. Site runs on localhost:3000
5. First Cursor prompt: project structure, `data/products.ts` , `lib/products.ts` , `lib/whatsapp.ts` , layout, header, footer
6. Home page (I saw the screenshot)
7. Collection page: you said "it built it"

Not confirmed

- **Product detail page** (`/collection/[slug]`): I gave you the prompt, but you never said you ran it. You asked about port forwarding right after.
- **Red "1 Issue" badge:** I never heard back whether it's gone.
- **`NEXT_PUBLIC_SITE_URL`** in `.env.local` , which that prompt needs.

How to check the product page

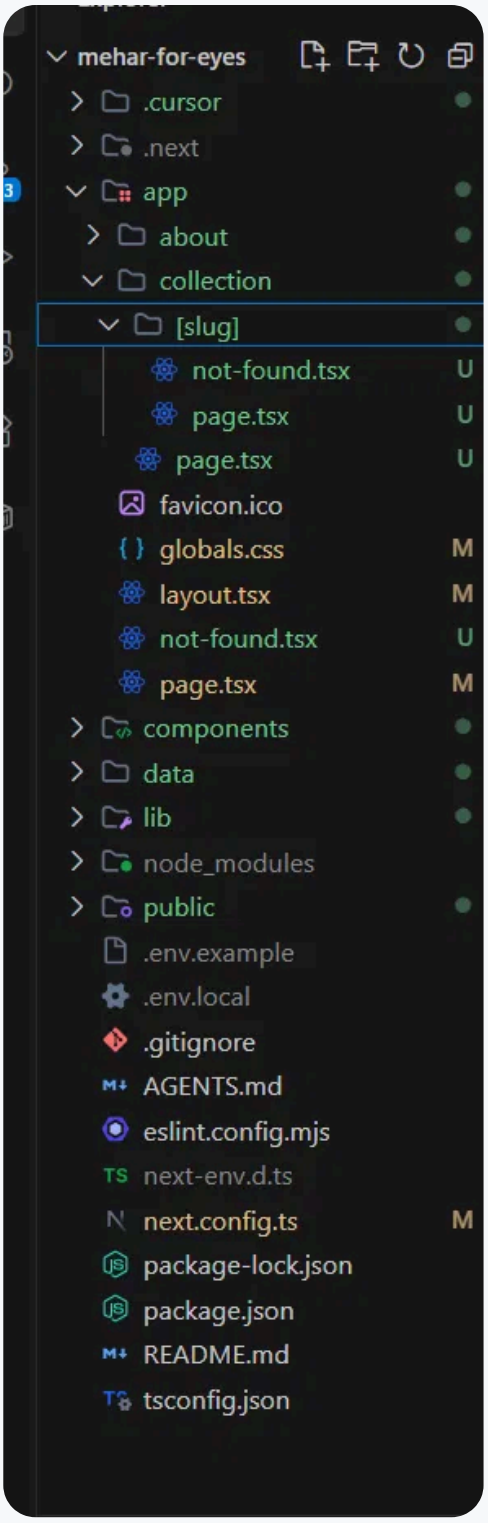
1. In the left file panel, open `app` then `collection`. If there's a folder named `[slug]` with a `page.tsx` inside, it was built. If not, you haven't run the prompt yet.
2. Or open `localhost:3000/collection` and click any product. A real product page means it's done. A 404 means it isn't.

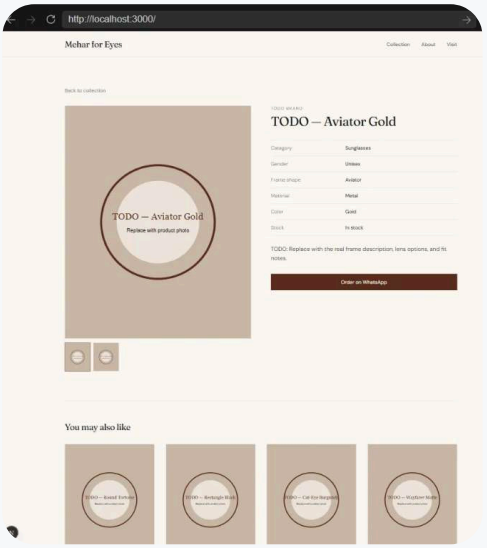
What to do

- **Product page exists:** check `.env.local` has `NEXT_PUBLIC_SITE_URL=http://localhost:3000`, restart with `npm run dev`, test the WhatsApp button, then send me a screenshot. Next up is the floating WhatsApp button.
- **It doesn't exist:** scroll up in this chat to my "Build the product detail page" message and paste that prompt into Cursor (Agent mode).

Also click the red badge if it's still there and send me the message.

2026-11-01 21:30:05 **You**


mehar-for-eyes
.cursor
.next
app
about
collection
[slug]
not-found.tsx U
page.tsx U
page.tsx U
favicon.ico
globals.css M
layout.tsx M
not-found.tsx U
page.tsx M
components
data
lib
node_modules
public
.env.example
.env.local
.gitignore
AGENTS.md
eslint.config.mjs
next-env.d.ts
next.config.ts M
package-lock.json
package.json
README.md
tsconfig.json


http://localhost:3000/
Mehar for Eyes
TODO — Aviator Gold
You may also like

see this

Claude

Product page is built and working. Gallery with thumbnails, specs table, WhatsApp button and "You may also like" are all there. The red "1 Issue" badge is gone too. Only the small N icon remains, which is normal.

Quick tests (2 minutes)

1. Click **Order on WhatsApp**. WhatsApp should open with a message that has the product name in it. If it opens a wrong number or an empty chat, tell me.
2. Type `localhost:3000/collection/abc` in the browser. You should see a not-found page, not a crash.
3. Some words in the text show in blue/red/orange (like "Unisex", "Replace", "lens options"). That's probably your browser or an extension coloring text, not your code. Open the page in normal Chrome and check. If the colors are still there, tell me.

Next: floating WhatsApp button

Paste into Cursor (Agent mode):

```
</> Plain text

Add a floating WhatsApp button to the whole site. Create components/WhatsAppFloat.tsx and add
it to app/layout.tsx.

- Fixed at bottom right, 56px solid circle in the accent color (#5A2E1F) with a simple white
WhatsApp logo as inline SVG. No pulsing, bouncing or glow animation. No shadow.
- Link uses buildWhatsAppLink with the message: "Hi Mehar for Eyes, I have a question."
- It opens in a new tab with rel="noopener noreferrer" and aria-label="Chat on WhatsApp".
- On the product detail page /collection/[slug], the mobile sticky bottom bar already has a
WhatsApp button, so hide the floating button there on mobile screens, and show it on desktop.
- Make sure it never covers the footer text or other buttons on a 360px screen.

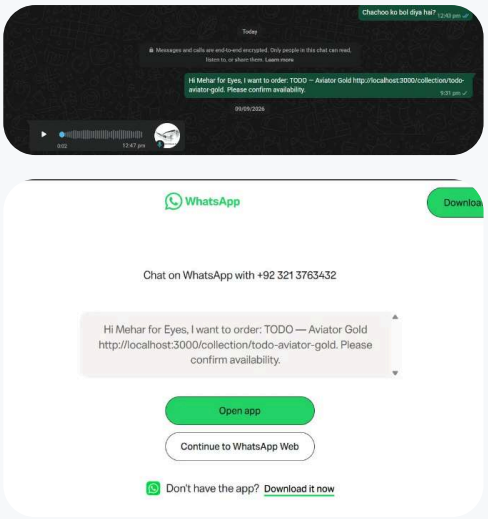
Follow the design rules in .cursor/rules. Return full files.
```

After that: About page and SEO

When the floating button works, tell me and I'll give you the next prompt. It covers the About page (story, address, hours, map), page titles, Google search tags, and the sitemap.

Send a screenshot when the button is done, ideally the mobile view (right-click, Inspect, then the phone icon at the top).

2026-11-01 21:31:50 **You**



dang it worked

Claude

Yes, it works. The page opened with the right number (+92 321 3763432) and the message filled in, product name and link included.

Two notes:

1. **That message really got sent.** Your chat screenshot shows it delivered at 9:31 pm. If that number is the shop's real WhatsApp, delete the test message from the chat before anyone sees "TODO — Aviator Gold" in there.
2. **The link says `localhost:3000`.** That's correct for now. When the site goes live, change `NEXT_PUBLIC_SITE_URL` to the real domain, or customers will get a link that only works on your PC.

Next step: floating WhatsApp button

Paste this into Cursor (Agent mode):

```
</> Plain text

Add a floating WhatsApp button to the whole site. Create components/WhatsAppFloat.tsx and add
it to app/layout.tsx.
```

```
- Fixed at bottom right, 56px solid circle in the accent color (#5A2E1F) with a simple white
WhatsApp logo as inline SVG. No pulsing, bouncing or glow animation. No shadow.
- Link uses buildWhatsAppLink with the message: "Hi Mehar for Eyes, I have a question."
- It opens in a new tab with rel="noopener noreferrer" and aria-label="Chat on WhatsApp".
- On the product detail page /collection/[slug], the mobile sticky bottom bar already has a
WhatsApp button, so hide the floating button there on mobile screens, and show it on desktop.
- Make sure it never covers the footer text or other buttons on a 360px screen.

Follow the design rules in .cursor/rules. Return full files.
```

Then check:

- The button shows on the home and collection pages, bottom right.
- On a product page in mobile view (right-click, Inspect, phone icon), you don't see two WhatsApp buttons stacked.
- Clicking it opens WhatsApp with "I have a question".

Send a screenshot of the mobile view when it's done. After that comes the About page and SEO.

2026-11-01 21:36:10 **You**

well the problem is my cursor ended so i am using antigravity with pony tail so maybe u have to focus on making a better prompt since antigravity is quite bad at making things

Claude

Fine. A weaker model needs smaller tasks, exact file names, and code written for it where possible. For small pieces like this, I'll write the code and Antigravity just places it.

1. Rules file in Antigravity

I'm not sure which rules file Antigravity auto-reads. Your project already has `AGENTS.md`, which many tools read. Do this:

- Open `AGENTS.md` and paste the full contents of your old `mehar.mdc` rules at the bottom.
- In the first prompt of each new chat, also write: "Read AGENTS.md and follow every rule in it."

2. A template for every prompt

Weak models fail when told "build X". Use this shape every time:

```
</> Plain text

TASK: [one small thing]

FIRST READ these files and do not change them: [list]

CREATE or EDIT only these files: [list]

REQUIREMENTS:
1. ...
2. ...

DO NOT: touch any other file, install packages, rename existing functions, add comments about
what you did.

WHEN DONE: run `npm run build`, fix any errors, then list the files you changed.
```

The "only these files" line stops the model from breaking things that already work. The `npm run build` line catches errors before you see them.

3. Floating WhatsApp button, with the code written for it

```
</> Plain text

TASK: Add a floating WhatsApp button to the whole site.

FIRST READ these files and do not change them: lib/whatsapp.ts, AGENTS.md

CREATE: components/WhatsAppFloat.tsx with exactly the code below.
EDIT: app/layout.tsx, only to import WhatsAppFloat and render <Suspense fallback={null}>
<WhatsAppFloat /></Suspense> just before the closing </body> tag. Import Suspense from
"react".

Code for components/WhatsAppFloat.tsx:

"use client";

import { usePathname } from "next/navigation";
import { buildWhatsAppLink } from "@/lib/whatsapp";

export default function WhatsAppFloat() {
  const pathname = usePathname();
  const isProductPage = /^\/collection\/[^/]+$/.test(pathname);
  const href = buildWhatsAppLink("Hi Mehar for Eyes, I have a question.");

  return (
```

```
    <a
      href={href}
      target="_blank"
      rel="noopener noreferrer"
      aria-label="Chat on WhatsApp"
      className={`fixed bottom-5 right-5 z-40 h-14 w-14 items-center justify-center rounded-
full bg-[#5A2E1F] text-white ${
        isProductPage ? "hidden md:flex" : "flex"
      }`}
    >
      <svg
        width="26"
        height="26"
        viewBox="0 0 24 24"
        fill="none"
        stroke="currentColor"
        strokeWidth="1.8"
        strokeLinecap="round"
        strokeLinejoin="round"
        aria-hidden="true"
      >
        <path d="M21 11.5a8.38 8.38 0 0 1-.9 3.8 8.5 8.5 0 0 1-7.6 4.7 8.38 8.38 0 0 1-
3.8-.9L3 21l1.9-5.7a8.38 8.38 0 0 1-.9-3.8 8.5 8.5 0 0 1 4.7-7.6 8.38 8.38 0 0 1
3.8-.9h.5a8.48 8.48 0 0 1 8 8v.5z" />
      </svg>
    </a>
  );
}

If buildWhatsAppLink in lib/whatsapp.ts has a different signature than (message: string) =>
string, adapt only the call inside WhatsAppFloat.tsx.

DO NOT: touch any other file, install packages, add shadows or animations.

WHEN DONE: run `npm run build`, fix any errors, then list the files you changed.
```

The icon is a simple chat bubble. You can swap in the official WhatsApp logo later.

4. Check the result

- The button shows bottom right on home and collection pages.
- On a product page in mobile view (right-click, Inspect, phone icon), the floating button is hidden because the sticky bar is there. On desktop width it shows.
- Click it and WhatsApp opens with "I have a question".

If Antigravity keeps messing up

- **One prompt, one chat.** Start a fresh chat for each task so old context doesn't confuse it.
- **If it breaks something,** use Git. Your project already has a repo. Before each prompt run `git add -A` and then `git commit -m "before next step"`. If the result is bad, run `git restore .` to go back.
- **Check which model it's using.** Antigravity lets you pick the model in the chat box. If a stronger one is available in your plan, use it for bigger steps like the admin dashboard and Go backend.
- **Paste me its mistakes.** Screenshot or paste the bad output and I'll write a fix prompt.

Next step after this: the About page and SEO. I'll write it in the same template with the code included where I can.

2026-11-01 22:24:26 **You**

nah dont divide prompt just make that prompt more detailed

Claude

Here is one single prompt that does everything left in the public site: floating WhatsApp button, About page, and SEO. It contains the code for the small files, so Antigravity only has to place them and connect them.

</> Plain text

```
TASK: Finish the public site in one pass: (A) floating WhatsApp button, (B) central site
config, (C) About page, (D) SEO: metadata, structured data, sitemap, robots.

FIRST READ these files and do not change them: AGENTS.md, lib/whatsapp.ts, lib/products.ts,
data/products.ts, app/page.tsx (to copy the exact style of its "Visit us" section: spacing,
headings, buttons).

ALLOWED FILES. Create: lib/site.ts, components/WhatsAppFloat.tsx,
components/LocalBusinessJsonLd.tsx, app/sitemap.ts, app/robots.ts. Edit: app/layout.tsx,
app/about/page.tsx, app/collection/page.tsx (metadata only), app/collection/[slug]/page.tsx
(generateMetadata title only). Touch NO other file. Install NO packages. Do not rename
existing functions or exports.

GLOBAL RULES (apply to everything below): follow the design rules in AGENTS.md. No shadows,
no gradients, no emojis, no generic marketing copy. Real-looking text that I must replace
later is written as "TODO - ..." exactly like the existing placeholders. Mobile first, must
look right at 360px. Keep Next.js 16 conventions already used in this project.

--- PART A: lib/site.ts ---
```

```
Create lib/site.ts with exactly this content:

const whatsappNumber = process.env.NEXT_PUBLIC_WHATSAPP_NUMBER ?? "";

export const siteConfig = {
  name: "Mehar for Eyes",
  description: "TODO — One or two sentences about the shop, its location and what it sells.",
  url: process.env.NEXT_PUBLIC_SITE_URL ?? "http://localhost:3000",
  whatsappNumber,
  phone: whatsappNumber ? "+" + whatsappNumber : "",
  address: {
    street: "TODO — Street address",
    locality: "TODO — City",
    region: "TODO — Province",
    country: "PK",
  },
  hours: "TODO — Day–Day, 00:00–00:00",
};

--- PART B: components/WhatsAppFloat.tsx ---
Create components/WhatsAppFloat.tsx with exactly this content:

"use client";

import { usePathname } from "next/navigation";
import { buildWhatsAppLink } from "@/lib/whatsapp";

export default function WhatsAppFloat() {
  const pathname = usePathname();
  const isProductPage = /^\/collection\/[^/]+$/.test(pathname);
  const href = buildWhatsAppLink("Hi Mehar for Eyes, I have a question.");

  return (
    <a
      href={href}
      target="_blank"
      rel="noopener noreferrer"
      aria-label="Chat on WhatsApp"
      className={`fixed bottom-5 right-5 z-40 h-14 w-14 items-center justify-center rounded-
full bg-[#5A2E1F] text-white ${
        isProductPage ? "hidden md:flex" : "flex"
      }`}
    >
      <svg width="26" height="26" viewBox="0 0 24 24" fill="none" stroke="currentColor"
strokeWidth="1.8" strokeLinecap="round" strokeLinejoin="round" aria-hidden="true">
        <path d="M21 11.5a8.38 8.38 0 0 1-.9 3.8 8.5 8.5 0 0 1-7.6 4.7 8.38 8.38 0 0 1-
3.8-.9L3 21l1.9-5.7a8.38 8.38 0 0 1-.9-3.8 8.5 8.5 0 0 1 4.7-7.6 8.38 8.38 0 0 1
3.8-.9h.5a8.48 8.48 0 0 1 8 8v.5z" />
      </svg>
```

```
    </a>
  );
}

If buildWhatsAppLink does not have the signature (message: string) => string, adapt only the
call inside this file.
The product detail page already has a sticky mobile bottom bar with a WhatsApp button, which
is why this button is hidden on mobile there. Make sure on a 360px screen the floating button
never covers footer text or any other button; if it does, add bottom padding to the footer,
not to the button.

--- PART C: components/LocalBusinessJsonLd.tsx ---
Create components/LocalBusinessJsonLd.tsx with exactly this content:

import { siteConfig } from "@/lib/site";

export default function LocalBusinessJsonLd() {
  const data = {
    "@context": "https://schema.org",
    "@type": "Optician",
    name: siteConfig.name,
    url: siteConfig.url,
    telephone: siteConfig.phone,
    address: {
      "@type": "PostalAddress",
      streetAddress: siteConfig.address.street,
      addressLocality: siteConfig.address.locality,
      addressRegion: siteConfig.address.region,
      addressCountry: siteConfig.address.country,
    },
  };

  return (
    <script
      type="application/ld+json"
      dangerouslySetInnerHTML={{ __html: JSON.stringify(data).replace(/</g, "\\u003c") }}
    />
  );
}

--- PART D: app/layout.tsx ---
Edit app/layout.tsx. Keep everything that already exists (fonts, header, footer, children).
Make only these changes:
1. Import Suspense from "react", WhatsAppFloat, LocalBusinessJsonLd, and siteConfig.
2. Replace or extend the exported metadata so it is exactly:
   metadataBase: new URL(siteConfig.url),
   title: { default: "Mehar for Eyes | Eyewear and optical shop", template: "%s | Mehar for
Eyes" },
   description: siteConfig.description,
```

```
   openGraph: { siteName: siteConfig.name, type: "website", locale: "en_PK" },
   Add a TODO comment next to openGraph saying: add an og image when real shop photos exist.
3. Just before the closing </body> tag render, in this order: <LocalBusinessJsonLd /> and
<Suspense fallback={null}><WhatsAppFloat /></Suspense>.

--- PART E: app/about/page.tsx ---
Rewrite app/about/page.tsx. Export metadata with title "About" (the template adds the site
name) and a description of one sentence. Page sections, in order, using the same container
width, heading font (Fraunces), spacing and hairline dividers as app/page.tsx:
1. Heading "About the shop" and a short intro paragraph: "TODO - Two or three sentences about
how long the shop has served the area and what it is known for."
2. "Our story": two paragraphs, both "TODO - ..." placeholders that I will replace with the
real story.
3. "What we offer": a plain list of 4 items, each a short "TODO - ..." line (for example
frames, prescription lenses, sunglasses, kids eyewear). No icons.
4. "Visit us": address and hours read from siteConfig (not hardcoded), a Call button (tel:
link using siteConfig.phone) and a WhatsApp button (buildWhatsAppLink with the message "Hi
Mehar for Eyes, I have a question."), and a map placeholder box with the text "TODO - Map
goes here" exactly like the one on the home page.
Buttons must look identical to the buttons on the home page.

--- PART F: metadata on other pages ---
- app/collection/page.tsx: add exported metadata with title "Collection" and description
"Browse eyeglass frames, sunglasses and kids eyewear from Mehar for Eyes. Order on WhatsApp."
If the page already uses generateMetadata or searchParams in a way that conflicts, keep it
working and only add the title and description.
- app/collection/[slug]/page.tsx: in generateMetadata, the title must be the product name
only (the layout template already adds " | Mehar for Eyes"). Do not repeat the shop name
there. Keep the description.

--- PART G: sitemap and robots ---
Create app/sitemap.ts with this content. If getProducts needs a different argument, adapt
only that call:

import type { MetadataRoute } from "next";
import { getProducts } from "@/lib/products";
import { siteConfig } from "@/lib/site";

export default async function sitemap(): Promise<MetadataRoute.Sitemap> {
  const products = await getProducts({});
  const now = new Date();
  const staticPages = ["", "/collection", "/about"].map((path) => ({
    url: siteConfig.url + path,
    lastModified: now,
  }));
  const productPages = products.map((p) => ({
    url: siteConfig.url + "/collection/" + p.slug,
    lastModified: now,
  }));
```

```
  return [...staticPages, ...productPages];
}

Create app/robots.ts with this content:

import type { MetadataRoute } from "next";
import { siteConfig } from "@/lib/site";

export default function robots(): MetadataRoute.Robots {
  return {
    rules: { userAgent: "*", allow: "/", disallow: ["/admin", "/api"] },
    sitemap: siteConfig.url + "/sitemap.xml",
  };
}

--- DO NOT ---
Touch any file outside the allowed list. Add animations, shadows, gradients or new colors.
Invent real addresses, phone numbers, hours or shop history. Remove existing code that is not
mentioned above. Leave unused imports.

--- WHEN DONE ---
1. Run `npm run build` and fix every error and warning you caused. Do not stop until the
build passes.
2. List every file you created or changed, one line each.
3. Tell me anything you could not do exactly as written and why.
```

How to check it worked

1. `localhost:3000` : the round brown WhatsApp button is bottom right. Click it, WhatsApp opens with "I have a question".
2. A product page on a mobile-width view (right-click, Inspect, phone icon): no floating button, only the sticky bar. On desktop width: floating button shows.
3. `localhost:3000/about` : the new About page, with TODO text.
4. `localhost:3000/sitemap.xml` : a list of your pages and products.
5. `localhost:3000/robots.txt` : shows the rules and the sitemap line.
6. Browser tab titles: "About | Mehar for Eyes", "Collection | Mehar for Eyes", and product pages show "TODO — Aviator Gold | Mehar for Eyes", not the shop name twice.

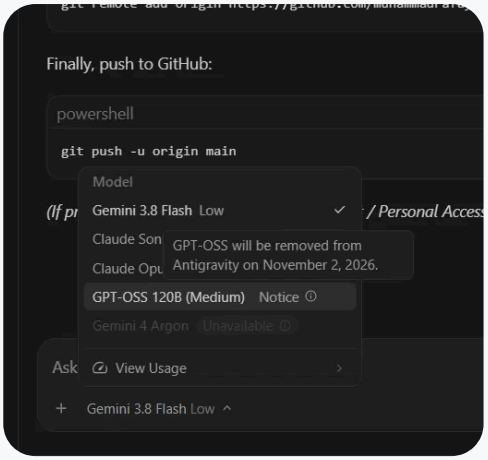
If `npm run build` fails or a page breaks, paste me the exact error and I'll write the fix.

Later, when real info exists

All shop details are in one file, `lib/site.ts` . Replace the TODO address, hours and description there and the About page, footer data and Google tags all update. Add opening hours to the structured data

once they're real, because a wrong format there is worse than none.

2026-11-01 22:30:20 **You**



which model to use for coding

Claude

Use **Claude Sonnet** for almost everything. Switch to **Claude Opus** only for the hard parts.

- **Gemini 3.8 Flash (Low)**: this is what you have selected now, and it's the weakest and fastest option on the list. "Low" also means it thinks the least. This is most likely why Antigravity feels bad at making things.
- **Claude Sonnet**: best default for this project. Good at following long, exact prompts like the ones I write, and good at not touching files it shouldn't.
- **Claude Opus**: slower and uses more of your quota. Save it for the Go backend, the auth, and the image upload security step, where a mistake costs more.
- **GPT-OSS 120B**: skip it. The notice says Antigravity removes it on November 2, which is tomorrow.
- **Gemini 4 Argon**: shows as unavailable.

I'm a Claude model, so weigh my recommendation accordingly. But Flash on Low is the weakest option in that list, so switching up is a safe move either way.

Before you rely on it, click **View Usage** at the bottom of that menu. The Claude options may have a limit on your plan, and Opus will use it up faster. If Sonnet runs out, fall back to Gemini Flash, but raise its setting from Low to a higher level if the option exists.

Then paste the big prompt I gave you, using Sonnet. If the result is still bad, send me the error or a screenshot.

2026-11-01 22:31:15 **You**

gimme the prompt again

Claude

Switch the model to **Claude Sonnet** first, start a fresh chat, then paste this:

</> Plain text

```
TASK: Finish the public site in one pass: (A) floating WhatsApp button, (B) central site
config, (C) About page, (D) SEO: metadata, structured data, sitemap, robots.

FIRST READ these files and do not change them: AGENTS.md, lib/whatsapp.ts, lib/products.ts,
data/products.ts, app/page.tsx (to copy the exact style of its "Visit us" section: spacing,
headings, buttons).

ALLOWED FILES. Create: lib/site.ts, components/WhatsAppFloat.tsx,
components/LocalBusinessJsonLd.tsx, app/sitemap.ts, app/robots.ts. Edit: app/layout.tsx,
app/about/page.tsx, app/collection/page.tsx (metadata only), app/collection/[slug]/page.tsx
(generateMetadata title only). Touch NO other file. Install NO packages. Do not rename
existing functions or exports.

GLOBAL RULES (apply to everything below): follow the design rules in AGENTS.md. No shadows,
no gradients, no emojis, no generic marketing copy. Real-looking text that I must replace
later is written as "TODO - ..." exactly like the existing placeholders. Mobile first, must
look right at 360px. Keep Next.js 16 conventions already used in this project.

--- PART A: lib/site.ts ---
Create lib/site.ts with exactly this content:

const whatsappNumber = process.env.NEXT_PUBLIC_WHATSAPP_NUMBER ?? "";

export const siteConfig = {
  name: "Mehar for Eyes",
  description: "TODO - One or two sentences about the shop, its location and what it sells.",
  url: process.env.NEXT_PUBLIC_SITE_URL ?? "http://localhost:3000",
  whatsappNumber,
  phone: whatsappNumber ? "+" + whatsappNumber : "",
  address: {
    street: "TODO - Street address",
```

```
    locality: "TODO - City",
    region: "TODO - Province",
    country: "PK",
  },
  hours: "TODO - Day-Day, 00:00-00:00",
};

--- PART B: components/WhatsAppFloat.tsx ---
Create components/WhatsAppFloat.tsx with exactly this content:

"use client";

import { usePathname } from "next/navigation";
import { buildWhatsAppLink } from "@/lib/whatsapp";

export default function WhatsAppFloat() {
  const pathname = usePathname();
  const isProductPage = /^\/collection\/[^/]+$/.test(pathname);
  const href = buildWhatsAppLink("Hi Mehar for Eyes, I have a question.");

  return (
    <a
      href={href}
      target="_blank"
      rel="noopener noreferrer"
      aria-label="Chat on WhatsApp"
      className={`fixed bottom-5 right-5 z-40 h-14 w-14 items-center justify-center rounded-
full bg-[#5A2E1F] text-white ${
        isProductPage ? "hidden md:flex" : "flex"
      }`}
    >
      <svg width="26" height="26" viewBox="0 0 24 24" fill="none" stroke="currentColor"
strokeWidth="1.8" strokeLinecap="round" strokeLinejoin="round" aria-hidden="true">
        <path d="M21 11.5a8.38 8.38 0 0 1-.9 3.8 8.5 8.5 0 0 1-7.6 4.7 8.38 8.38 0 0 1-
3.8-.9L3 21l1.9-5.7a8.38 8.38 0 0 1-.9-3.8 8.5 8.5 0 0 1 4.7-7.6 8.38 8.38 0 0 1
3.8-.9h.5a8.48 8.48 0 0 1 8 8v.5z" />
      </svg>
    </a>
  );
}

If buildWhatsAppLink does not have the signature (message: string) => string, adapt only the
call inside this file.
The product detail page already has a sticky mobile bottom bar with a WhatsApp button, which
is why this button is hidden on mobile there. Make sure on a 360px screen the floating button
never covers footer text or any other button; if it does, add bottom padding to the footer,
not to the button.

--- PART C: components/LocalBusinessJsonLd.tsx ---
```

```
Create components/LocalBusinessJsonLd.tsx with exactly this content:

import { siteConfig } from "@/lib/site";

export default function LocalBusinessJsonLd() {
  const data = {
    "@context": "https://schema.org",
    "@type": "Optician",
    name: siteConfig.name,
    url: siteConfig.url,
    telephone: siteConfig.phone,
    address: {
      "@type": "PostalAddress",
      streetAddress: siteConfig.address.street,
      addressLocality: siteConfig.address.locality,
      addressRegion: siteConfig.address.region,
      addressCountry: siteConfig.address.country,
    },
  };

  return (
    <script
      type="application/ld+json"
      dangerouslySetInnerHTML={{ __html: JSON.stringify(data).replace(/</g, "\\u003c") }}
    />
  );
}

--- PART D: app/layout.tsx ---
Edit app/layout.tsx. Keep everything that already exists (fonts, header, footer, children).
Make only these changes:
1. Import Suspense from "react", WhatsAppFloat, LocalBusinessJsonLd, and siteConfig.
2. Replace or extend the exported metadata so it is exactly:
   metadataBase: new URL(siteConfig.url),
   title: { default: "Mehar for Eyes | Eyewear and optical shop", template: "%s | Mehar for
Eyes" },
   description: siteConfig.description,
   openGraph: { siteName: siteConfig.name, type: "website", locale: "en_PK" },
   Add a TODO comment next to openGraph saying: add an og image when real shop photos exist.
3. Just before the closing </body> tag render, in this order: <LocalBusinessJsonLd /> and
<Suspense fallback={null}><WhatsAppFloat /></Suspense>.

--- PART E: app/about/page.tsx ---
Rewrite app/about/page.tsx. Export metadata with title "About" (the template adds the site
name) and a description of one sentence. Page sections, in order, using the same container
width, heading font (Fraunces), spacing and hairline dividers as app/page.tsx:
1. Heading "About the shop" and a short intro paragraph: "TODO - Two or three sentences about
how long the shop has served the area and what it is known for."
2. "Our story": two paragraphs, both "TODO - ..." placeholders that I will replace with the
```

```
real story.
3. "What we offer": a plain list of 4 items, each a short "TODO - ..." line (for example
frames, prescription lenses, sunglasses, kids eyewear). No icons.
4. "Visit us": address and hours read from siteConfig (not hardcoded), a Call button (tel:
link using siteConfig.phone) and a WhatsApp button (buildWhatsAppLink with the message "Hi
Mehar for Eyes, I have a question."), and a map placeholder box with the text "TODO - Map
goes here" exactly like the one on the home page.
Buttons must look identical to the buttons on the home page.

--- PART F: metadata on other pages ---
- app/collection/page.tsx: add exported metadata with title "Collection" and description
"Browse eyeglass frames, sunglasses and kids eyewear from Mehar for Eyes. Order on WhatsApp."
If the page already uses generateMetadata or searchParams in a way that conflicts, keep it
working and only add the title and description.
- app/collection/[slug]/page.tsx: in generateMetadata, the title must be the product name
only (the layout template already adds " | Mehar for Eyes"). Do not repeat the shop name
there. Keep the description.

--- PART G: sitemap and robots ---
Create app/sitemap.ts with this content. If getProducts needs a different argument, adapt
only that call:

import type { MetadataRoute } from "next";
import { getProducts } from "@/lib/products";
import { siteConfig } from "@/lib/site";

export default async function sitemap(): Promise<MetadataRoute.Sitemap> {
  const products = await getProducts({});
  const now = new Date();
  const staticPages = ["", "/collection", "/about"].map((path) => ({
    url: siteConfig.url + path,
    lastModified: now,
  }));
  const productPages = products.map((p) => ({
    url: siteConfig.url + "/collection/" + p.slug,
    lastModified: now,
  }));
  return [...staticPages, ...productPages];
}

Create app/robots.ts with this content:

import type { MetadataRoute } from "next";
import { siteConfig } from "@/lib/site";

export default function robots(): MetadataRoute.Robots {
  return {
    rules: { userAgent: "*", allow: "/", disallow: ["/admin", "/api"] },
    sitemap: siteConfig.url + "/sitemap.xml",
```

```
  };
}

--- DO NOT ---
Touch any file outside the allowed list. Add animations, shadows, gradients or new colors.
Invent real addresses, phone numbers, hours or shop history. Remove existing code that is not
mentioned above. Leave unused imports.

--- WHEN DONE ---
1. Run `npm run build` and fix every error and warning you caused. Do not stop until the
build passes.
2. List every file you created or changed, one line each.
3. Tell me anything you could not do exactly as written and why.
```

Before you paste it, save your current work so you can undo if it goes wrong. In the terminal:

```
</> Plain text

git add -A
git commit -m "before about and seo"
```

If the result is bad, run `git restore .` and you're back where you started.

Checks afterwards:

1. Home page: brown round WhatsApp button, bottom right.
2. `localhost:3000/about` : new About page with TODO text.
3. `localhost:3000/sitemap.xml` and `localhost:3000/robots.txt` both show content.
4. Product page in mobile view: no floating button, only the sticky bar.

Paste me any error exactly as shown.

2026-11-01 22:41:01 **You**

ok it did now how to check

Claude

Do these checks one by one. Make sure `npm run dev` is running first (restart it with Ctrl+C then `npm run dev` if unsure).

1. Home page

Open `http://localhost:3000`

- You should see a **round brown button** at the bottom right.
- Click it. WhatsApp opens with the message "Hi Mehar for Eyes, I have a question."

2. About page

Open `http://localhost:3000/about`

- You should see: About the shop, Our story, What we offer, Visit us. All text starts with "TODO".
- Click the Call and WhatsApp buttons. Both should work.
- The tab title should say **About | Mehar for Eyes**.

3. Sitemap and robots

Open these two links:

- `http://localhost:3000/sitemap.xml` : should show a list of links (home, collection, about, and every product)
- `http://localhost:3000/robots.txt` : should show a few lines, ending with a Sitemap line

4. Product page on mobile

1. Open any product page.
2. Right-click, choose **Inspect**, and click the **phone icon** at the top left of the panel.
3. Pick a small phone size (like 360 wide).
4. You should see only the sticky WhatsApp bar at the bottom, **not** the round button too.
5. Make the width wide again (desktop). The round button should show.

5. Tab titles

Look at the browser tab on each page:

- Collection page: **Collection | Mehar for Eyes**
- Product page: **TODO — Aviator Gold | Mehar for Eyes**. If the shop name shows twice, tell me.

6. Build check

In the terminal, stop the server (Ctrl+C) and run:

```
</> Plain text
```

```
npm run build
```

It should end without red errors. Then start `npm run dev` again.

Save your work

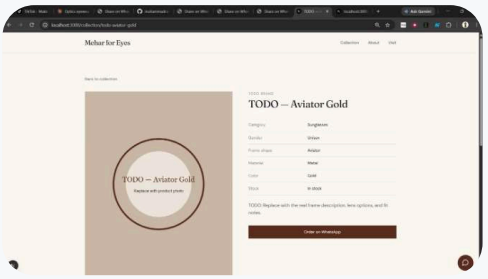
If everything looks fine:

```
</> Plain text

git add -A
git commit -m "about, seo, whatsapp float"
```

Tell me which checks pass and which fail. For a failure, send a screenshot or the exact error. If everything passes, the public site is done and the next step is the Go backend and admin dashboard (the real photo uploads). Before that, one question: do you have a VPS or server to host the backend, or should I plan it for a free or cheap host?

2026-11-01 22:50:46 **You**



i dont see any phone con on the top left panel, whiel the 1 2 adn 3 are totally correct and implemente

Claude

Your screenshot already shows the right result for desktop width: the round brown button at the bottom right on the product page. The browser tab titles (3 and 5) are fine too.

Easy way to open mobile view in Chrome

1. Press **F12** to open the developer panel.
2. Press **Ctrl + Shift + M**. This turns on mobile view, so you don't need to find the icon.

If you want to find the icon by hand: it's in the top-left corner of the developer panel (not the browser), a small icon of a phone and tablet overlapping. If the panel is docked at the bottom or right, look at the very first row of that panel.

If it still doesn't show

Make the browser window narrow instead. Drag the edge of the Chrome window until it is about 400px wide. The site responds to the real width, so it behaves the same as in phone view.

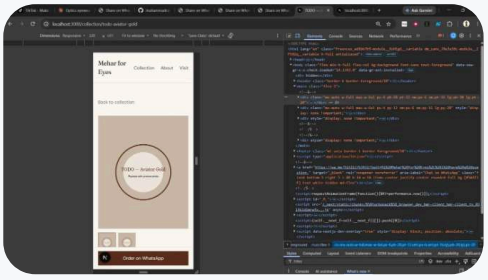
Or open `http://localhost:3000` on your actual phone. Your PC and phone must be on the same Wi-Fi. Find your PC's IP with `ipconfig` in the terminal (look for IPv4 Address, like `192.168.1.5`), then open `http://192.168.1.5:3000` on the phone. If it doesn't load, Windows Firewall may be blocking it, so skip this one.

What to check in mobile view

1. On the **product page**: only the sticky WhatsApp bar at the bottom, **no** round button.
2. On the **home page** and **about page**: the round button shows, and it doesn't cover any text or buttons.
3. Nothing sideways-scrolls (no horizontal scrollbar).

Send me a screenshot of the product page in mobile view if you can. If it looks right, the public site is done and we move to the Go backend and admin dashboard.

2026-11-01 23:33:05 **You**



is this the dev panne

Claude

Yes, that's the Chrome dev panel, and you already have mobile view on. The top bar says **Responsive 320 x 689**, and the site is shown narrow like a phone.

Result: pass

- The sticky **Order on WhatsApp** bar shows at the bottom.
- The round floating button is hidden. In the code on the right you can see `hidden md:flex` on it. That is correct.
- The small dark "N" circle on the left of the bar is Next.js's dev badge. It only shows in development, not on the live site.

Two small things to fix

1. The logo wraps onto two lines ("Mehar for / Eyes") at 320px, and the nav links look squeezed. Most phones are wider than 320, but fix it anyway. Paste this into Antigravity (Claude Sonnet):

```
</> Plain text

TASK: Fix the header on very narrow screens.

FIRST READ: AGENTS.md and the header component (find it from app/layout.tsx).
EDIT only the header component. Touch no other file.

REQUIREMENTS:
1. At 320px width the logo text "Mehar for Eyes" must stay on one line (whitespace-nowrap)
and the three nav links (Collection, About, Visit) must fit on the same row without wrapping.
2. Below the sm breakpoint: reduce the logo font size slightly, reduce the gap between nav
links, and reduce horizontal padding. At sm and above, keep the current look exactly.
3. No horizontal scrolling at any width from 320px up.

DO NOT: add a hamburger menu, install packages, change colors or fonts.

WHEN DONE: run `npm run build`, fix errors, list files changed.
```

2. The red "1" in the top right of the dev panel means there's one error somewhere. Click **Console** (the tab at the bottom left of the panel) and read the red line. Send me a screenshot or the exact text. It may just be a Chrome extension, since you have several installed, but check.

After that

The public site is done. Next is the Go backend and admin dashboard where your father uploads photos. Answer one thing so I plan it right: **do you have a VPS or server to host the Go backend, or should I plan around a free or cheap host?**

2026-11-02 23:59:57 **You**

so where were we last

Claude

Done

1. Rules file, Next.js project, `.env.local` with the WhatsApp number
2. Home page, collection page with filters, product page
3. Floating WhatsApp button, About page, SEO (page titles, sitemap, robots, business data)
4. Mobile check: the product page shows the sticky WhatsApp bar and hides the round button, which is correct

All of it runs on placeholder products and TODO text.

Where we stopped

Your last screenshot showed the mobile view of the product page. Two small things were left open:

1. **Header wraps at 320px.** The logo "Mehar for Eyes" breaks onto two lines on very narrow screens. I gave you a prompt for Antigravity to fix it. Did you run it?
2. **Red "1" in the dev panel.** It means one error in the browser console. Click the **Console** tab, read the red line, and send it to me. It may just be one of your Chrome extensions.

Also on your list

- **Commit your work** so you can undo later:

```
</> Plain text

git add -A
git commit -m "about, seo, whatsapp float"
```

- **Delete the test message** from the shop's WhatsApp chat (the "TODO — Aviator Gold" one), if that number is the real shop number.
- **Check that `NEXT_PUBLIC_SITE_URL` is in `.env.local`.** It stays `http://localhost:3000` until you launch.

Next big step

The Go backend and admin dashboard, where your father uploads real photos and manages products. One question before I write those prompts: **do you have a VPS or server for hosting the Go backend, or should I plan around a free or cheap host?** Also tell me whether you did the header fix.